

Chapter 23: Kotlin Coroutines

By Irina Galata

While working through the previous chapters, you've run **synchronous** code only. That means that one command was executed after another by your CPU, sequentially, and no code in your projects were running simultaneously on different computing cores (in the case that your CPU has them, which they tend to these days).

Consequently, if you decided to perform any long-running, time-consuming operations (e.g., sending a request over a network to a server, or processing a large file), your program would appear to freeze until the operation finished, and a user would have to wait. That's less than ideal — a user should be able to interact with your program even while it's executing a difficult task. That expectation leads to the concept of the *asynchronous programming*.

Asynchronous programming

As opposed to the synchronous approach, **asynchronous programming** allows for the execution of several tasks in parallel at the same time. That way, you can render a beautiful loader animation while your app is also retrieving the necessary data from a server, for example. Or you could break up a non-trivial task into a few easier ones and execute them simultaneously to decrease the processing time.

Threads

In Java — and accordingly in Kotlin on the JVM — you can parallelize your program using **threads**. Each `java.lang.Thread` object represents one execution flow, which sequentially performs the commands within the single thread.

You can operate on threads in various ways — create them, start, pause, join, etc. By creating several threads, you can perform multiple tasks simultaneously.

Take a look at the example below:

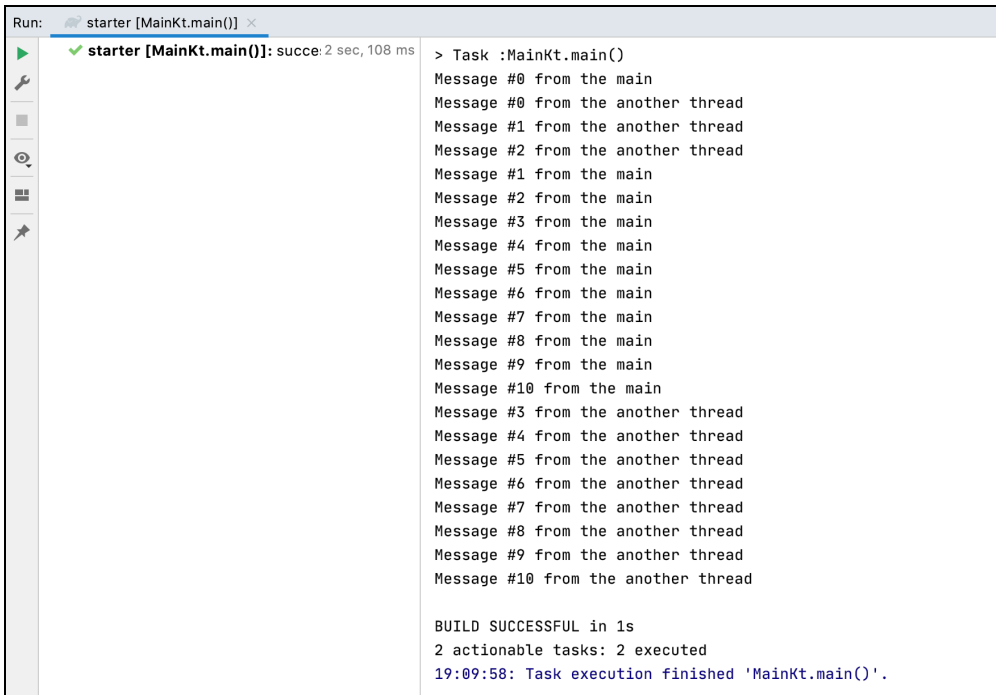
```
fun main() {
    thread(start = true, name = "another thread") {
        (0..10).forEach {
            println(
                "Message #${it} from the ${Thread.currentThread().name}")
        }
    }

    (0..10).forEach {
        println(
            "Message #${it} from the ${Thread.currentThread().name}")
    }
}
```

In the above, you first create a thread named “another thread” using the function `thread()` from the `kotlin.concurrent` package. You pass `true` for the `start` parameter, so the thread will start executing commands immediately. A message with the thread name and a number will be printed 11 times.

In the code below that, you perform the same work on the main, default thread for your project, without creating a new one.

If you run the code below, you'll see a similar output:



```
Run: starter [MainKt.main()] x
✓ starter [MainKt.main()]: succe: 2 sec, 108 ms

> Task :MainKt.main()
Message #0 from the main
Message #0 from the another thread
Message #1 from the another thread
Message #2 from the another thread
Message #1 from the main
Message #2 from the main
Message #3 from the main
Message #4 from the main
Message #5 from the main
Message #6 from the main
Message #7 from the main
Message #8 from the main
Message #9 from the main
Message #10 from the main
Message #3 from the another thread
Message #4 from the another thread
Message #5 from the another thread
Message #6 from the another thread
Message #7 from the another thread
Message #8 from the another thread
Message #9 from the another thread
Message #10 from the another thread

BUILD SUCCESSFUL in 1s
2 actionable tasks: 2 executed
19:09:58: Task execution finished 'MainKt.main()'.
```

The exact ordering of the parallel `println()` results from the two threads is indeterminate, and it depends on whatever is going on in your CPU at the time you run. You can see that your main thread, along with “another thread,” are executing at the same time without waiting for each other to complete, which is expected and is the desired behavior.

Everything seems fine with threads until you need to manipulate a large number of them or pass data back and forth between them. Also, it's important to note that Java threads are based on OS-level threads and, therefore, consume a significant amount of system resources. You can't create thousands of threads as you'll likely end up with an `OutOfMemoryError` thrown by the JVM.

Is there any other option, then?

Coroutines

There isn't an immediate better option in the Java language but, in Kotlin, you receive **coroutines** right out of the box! A coroutine is primarily a *computation*. Its defining feature is that it can be *suspended* and *resumed* at specified points of the computation without blocking a thread. Suspension is an extremely efficient operation. You can create hundreds and even thousands of coroutines and run them concurrently, as they are lightweight and don't require many extra resources for their execution.

Coroutines can be suspended at specified suspension points. These points are calls to functions marked with the suspend modifier. These suspending functions can only be invoked from coroutines or other suspending functions, as well as functions inlined in either coroutines or suspending routines.

Getting started

Open up the starter project for this chapter. The starter project contains a non-coroutine version of the example project you'll build below using coroutines. The `main()` function in **main.kt** looks as follows:

```
fun main() {  
    BuildingYard.startProject("Smart house", 20)  
}
```

Go ahead and run the `main()` function the same way you've done in previous chapters. You'll see a virtual building being constructed in the console.

One thing you'll notice is that it takes a *long* time to construct the building in a sequential manner, with each task being done one after another. When we switch to using coroutines in our project below, you'll see how asynchronous code makes the virtual building construction go much, much faster!

To get started with coroutines, add dependencies `{}` block and the coroutine dependency to your **build.gradle.kts** file:

```
dependencies {  
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:  
1.4.0")  
}
```

After that, you can write the earlier example like this:

```
fun main() = runBlocking {
    launch(Dispatchers.Default) {
        (0..10).forEach {
            println("Message #${it} from the $
{Thread.currentThread().name}")
        }

        (0..10).forEach {
            println("Message #${it} from the $
{Thread.currentThread().name}")
        }
    }
}
```

You will need to include the following imports to make the code above compile:

```
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.runBlocking
```

That code is the coroutine analog of the thread code at the beginning of this chapter. If you run this code, you'll get a similar result:

Note: You should remember that, when using coroutines, threads are still used under the hood. But one thread can execute thousands of coroutines. Therefore, you don't spend precious memory resources to manipulate a large number of coroutines.

Configuring coroutines

Kotlin coroutines are an extremely flexible solution for the wide variety of cases you may have. And the way a coroutine behaves is pretty much defined by its *context*. Any coroutine gets executed inside some `CoroutineScope` containing an instance of `CoroutineContext`, which is represented by a collection containing important configurations. You're going to get acquainted with the most important of them - `Job`, `Dispatcher` and, later in this chapter, `CoroutineExceptionHandler`.

Job

`Job` basically represents a background job, which has a state (active, cancelled, completed, etc.), optionally has children, and can be started and cancelled. You'll learn more about `Job` in this chapter.

Dispatchers

Dispatchers are responsible for the threads where your coroutines are executed. There are some ready-to-use dispatchers in the Kotlin core library:

1. `Dispatchers.Default` uses a pool of background threads for resource-demanding operations. The number of threads is equal to the number of cores on your machine, but at least two if it's a single-core CPU, which is highly unlikely nowadays.
2. `Dispatchers.IO` is useful when you need to perform input/output operations, e.g., saving user data to local storage or uploading files to a server. Use this dispatcher when a thread is supposed to be blocked while waiting for a response. It uses a pool of 64 threads.

3. `Dispatchers.Unconfined` is not limited to any thread. Don't use it unless you're sure that other dispatchers don't fit your case.
4. Optionally, you may use the single-threaded `Dispatchers.Main` for a UI-related Kotlin library (Android, JavaFx or Swing). You'll use it to perform operations on the UI thread and access UI objects.

CoroutineScope

`CoroutineScope` is an interface which does nothing except provide an associated `CoroutineContext`:

```
public interface CoroutineScope {  
    public val coroutineContext: CoroutineContext  
}
```

It's necessary to bind your coroutines to some *lifecycle* (if you're familiar with Android, the lifecycle of an Activity is a great example). That way, all jobs get cancelled as soon as your component/program completes and they're not necessary anymore.

Obtaining a scope

There are multiple ways to get `CoroutineScope` to launch a coroutine. Some of them are mentioned here:

1. Using `GlobalScope`, which is accessible from anywhere in your code. You can use it to execute top-level coroutines that shouldn't be bound to the lifecycle of some specific component, but rather the whole application. Always consider the options below before using this one.
2. The `MainScope()` function returns a scope which, like the `Main` dispatcher, is handy when you work with UI components.
3. You can use the `CoroutineScope(context: CoroutineContext)` function to wrap a specific context (Hint: You can use the dispatchers mentioned above, as `CoroutineDispatcher` is a `CoroutineContext` too).

Coroutines builders

In order to use coroutines and therefore parallelize the execution of your code, you need to use *coroutine builders*. They're regular functions that create a new coroutine inside a specified `CoroutineContext`. You've already seen some of them in the code snippet above — `runBlocking()` and `launch()`. Let's find out how they work.

`runBlocking()`

The declaration of the `runBlocking()` function in the coroutine library code is as follows:

```
public fun <T> runBlocking(  
    context: CoroutineContext = EmptyCoroutineContext,  
    block: suspend CoroutineScope.() -> T  
) : T
```

`runBlocking()` is a regular, non-coroutine function that creates a new coroutine to execute the *suspending* lambda you pass it as the parameter `block`. It *blocks* the current thread until the new coroutine execution finishes. That way, program execution won't stop and the coroutine will have time to complete. It's supposed to be used for testing purposes and in the `main()` function; in any other case, use other mentioned functions in order to avoid thread blocking and to use all the benefits of Kotlin coroutines.

`launch()`

The example code also used the `launch()` function, which has the following signature:

```
public fun CoroutineScope.launch(  
    context: CoroutineContext = EmptyCoroutineContext,  
    start: CoroutineStart = CoroutineStart.DEFAULT,  
    block: suspend CoroutineScope.() -> Unit  
) : Job
```

Similar to `runBlocking()` this function creates a new coroutine, but it doesn't block the current thread. Instead, it returns a `Job` object, which lets you control your coroutine execution. In the example above, you were not interested in sequential execution of the code, but parallel.

In case you need to wait for the execution of your newly created coroutine, you use the `join()` method of `Job` to *suspend* the current coroutine/suspend function until the job is done:

```
public suspend fun join()
```

That way you can call the `join()` function on a coroutine to wait until the result is ready:

```
launch { postVideoToFeed() }.join()
```

Note: All exceptions thrown during execution of a coroutine created with `launch()` are treated as *uncaught* exceptions and will fail the parent coroutine.

CoroutineStart

As you can see from the `launch` function declaration, you can specify not only the threads on which your coroutine will be launched, but also the moment when it should happen. There are four options:

1. `DEFAULT` corresponds to the immediate start of a coroutine.
2. `LAZY` — a coroutine won't be launched until it's necessary. You can do so by calling `start()` on the corresponding `Job` (or `Deferred`) object.
3. `ATOMIC` is similar to the default one, but the coroutine is not cancellable in that case.
4. If you use `UNDISPATCHED` start, the coroutine will be launched immediately until its first suspension point in the current thread.

async()

There will be numerous cases where you are interested not only in waiting for the coroutine to be executed, but also in getting a *result* from it. The most common case is getting data from a server—for example, loading a user profile or getting a list of chat messages. `async()` is a definite solution for this case:

```
public fun <T> CoroutineScope.async(  
    context: CoroutineContext = EmptyCoroutineContext,  
    start: CoroutineStart = CoroutineStart.DEFAULT,  
    block: suspend CoroutineScope.() -> T  
) : Deferred<T>
```

It's quite similar to `launch()`, but it returns a `Deferred` object, which is actually a `Job` itself (interface `Deferred` extends the `Job` interface), but it contains a *result* of the execution. In order to wait for the result, use the `await()` function:

```
public suspend fun await(): T
```

The `await()` function suspends the coroutine where this function is invoked until the result is ready and returns it without blocking the current thread:

```
val userData = async { getUserDataFromServer() }.await()
```

withContext()

The `withContext()` function gets the result of the execution as well. However, it's optimized for more straightforward cases, when you don't need the `Deferred` instance but just the *result* itself:

```
public suspend fun <T> withContext(  
    context: CoroutineContext,  
    block: suspend CoroutineScope.() -> T  
) : T
```

`withContext()` switches the coroutine to the specified context and *suspends* until the block is executed and the result of the job is available.

The various different builder functions associated with coroutines and their usage may seem overwhelming at first, and they're best understood by looking at an example.

Example: A high-rise building

To illustrate all the niceties of coroutines, it's necessary to imagine a process or task, some parts of which could be executed simultaneously, while other parts should be completed strictly one after another. The process of constructing a high-rise building is a good example.

The starter project contains a class named `Building` that represents the activities involved in building a new high-rise:

```
class Building(val name: String) {  
    fun makeFoundation() {  
        Thread.sleep(300)  
        speakThroughBullhorn("The foundation is ready")  
    }  
  
    fun buildFloor(floor: Int) {  
        Thread.sleep(100)  
        speakThroughBullhorn("The $floor'th floor is raised")  
    }  
  
    fun placeWindows(floor: Int) {  
        Thread.sleep(100)  
        speakThroughBullhorn("Windows are placed on the $floor'th  
floor")  
    }  
  
    fun installDoors(floor: Int) {  
        Thread.sleep(100)  
        speakThroughBullhorn("Doors are installed on the $floor'th  
floor")  
    }  
  
    fun provideElectricity(floor: Int) {  
        Thread.sleep(100)  
        speakThroughBullhorn("Electricity is provided on the  
$floor'th floor")  
    }  
  
    fun buildRoof() {  
        Thread.sleep(200)  
        speakThroughBullhorn("The roof is ready")  
    }  
  
    fun fitOut(floor: Int) {  
        Thread.sleep(200)  
        speakThroughBullhorn("The $floor'th floor is furnished")  
    }  
  
    fun speakThroughBullhorn(message: String) = println(message)  
}
```

In each function in `Building` in the starter project, we sleep the current thread for a certain number of milliseconds, and then call `speakThroughBullhorn()` to print a message.

To switch from using threads to working with coroutines and the associated functions, make the following changes to the `Building` class:

- Mark all functions except `speakThroughBullhorn()` in `Building` with the `suspend` modifier so that they can be called from coroutines and other suspending functions
- Wrap the body of each function except `speakThroughBullhorn()` with the `launch` function
- Change the `Thread.sleep()` calls to instead be the coroutine function `delay()`
- Prepend the strings that are printed with `[${Thread.currentThread().name}]`
- Add a `var floors: Int = 0` parameter to the `Building` constructor
- Increment the floor count using `++floors` at the end of the `buildFloor()` function

The result should be the following:

```
class Building(  
    val name: String,  
    var floors: Int = 0,  
    private val scope: CoroutineScope  
) {  
  
    suspend fun makeFoundation() = scope.launch {  
        delay(300)  
        speakThroughBullhorn("[${Thread.currentThread().name}] The  
foundation is ready")  
    }  
  
    suspend fun buildFloor(floor: Int) = scope.launch {  
        delay(100)  
        speakThroughBullhorn("[${Thread.currentThread().name}] Floor  
number $floor floor is built")  
        ++floors  
    }  
  
    suspend fun placeWindows(floor: Int) = scope.launch {  
        delay(100)  
        speakThroughBullhorn("[${Thread.currentThread().name}]  
Windows are placed on floor number $floor")  
    }  
}
```

```
suspend fun installDoors(floor: Int) = scope.launch {
    delay(100)
    speakThroughBullhorn("[${Thread.currentThread().name}] Doors
are installed on floor number $floor")
}

suspend fun provideElectricity(floor: Int) = scope.launch {
    delay(100)
    speakThroughBullhorn("[${Thread.currentThread().name}]
Electricity is provided on floor number $floor")
}

suspend fun buildRoof() = scope.launch {
    delay(200)
    speakThroughBullhorn("[${Thread.currentThread().name}] The
roof is ready")
}

suspend fun fitOut(floor: Int) = scope.launch {
    delay(200)
    speakThroughBullhorn("[${Thread.currentThread().name}] Floor
number $floor is furnished")
}

fun speakThroughBullhorn(message: String) = println(message)
}
```

In the `Building` class, you have functions that represent single tasks that should be completed during the building process.

For each of the tasks, you need a new coroutine to optimize the process. As you don't need a result from these tasks, you use the `launch()` function to create the coroutine. And, much like in the real world, a task can take some time to complete. You simulate waiting using the `delay()` function, which just suspends a coroutine for a specific amount of time. To build the high-rise, you need some physical space to place it. Update the contents of the file **BuildingYard.kt** from the starter project with a `BuildingYard` class that has a suspending function `startProject()`:

```
class BuildingYard {
    suspend fun startProject(name: String, floors: Int) {

    }
}
```

You initiate the process of building a twenty-floor high-rise in the `main()` function in `main.kt` as follows:

```
fun main() = runBlocking {  
    BuildingYard().startProject("Smart house", 20)  
}
```

As you don't want your program to shut down before the high-rise is ready, use the `runBlocking()` function.

Now, it's time to start the planning stage of building. Which task should come first? It's necessary to prepare the foundation, as it's an essential phase before starting any other one. Update the `startProject()` function in `BuildingYard` class as follows:

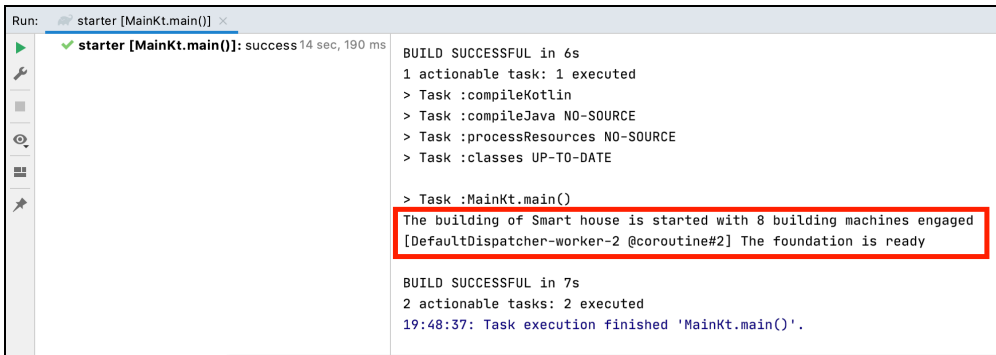
```
suspend fun startProject(name: String, floors: Int) {  
    val building = withContext(Dispatchers.Default) {  
        val building = Building(name, scope = this)  
        val cores = Runtime.getRuntime().availableProcessors()  
        building.speakThroughBullhorn(  
            "The building of $name is started with $cores building  
machines engaged")  
        building.makeFoundation().join()  
        building  
    }  
    if (building.floors == floors) {  
        building.speakThroughBullhorn("${building.name} is ready!")  
    }  
}
```

In entering the above, you expect to get a completed building as a result, so you wrap the whole building process in a lambda to pass it to `async()` and then call `await()` to suspend the current coroutine and wait for the result.

The `availableProcessors()` function on the `Runtime` returns the number of cores in the CPU of your computer. A *core* is responsible for performing the operations on the CPU. You have probably heard the term *multi-core processor*; this means that the CPU can perform multiple operations simultaneously. It's not uncommon for processors to have four cores or even eight. Don't worry though, you can still have more threads than cores as multiple threads can run on the same core!

You use the `join()` function in order to wait until the foundation is ready, as any other phase couldn't be started before that.

If you run the project now, you'll get the following result:



```
Run: starter [MainKt.main()] ×
✓ starter [MainKt.main()]: success 14 sec, 190 ms

BUILD SUCCESSFUL in 6s
1 actionable task: 1 executed
> Task :compileKotlin
> Task :compileJava NO-SOURCE
> Task :processResources NO-SOURCE
> Task :classes UP-TO-DATE

> Task :MainKt.main()
The building of Smart house is started with 8 building machines engaged
[DefaultDispatcher-worker-2 @coroutine#2] The foundation is ready

BUILD SUCCESSFUL in 7s
2 actionable tasks: 2 executed
19:48:37: Task execution finished 'MainKt.main()'.

```

The first line of output will be report the number of cores in your CPU.

With the foundation of the building ready, now it's possible to start working on the floors. Update the `startProject()` function to add a loop over the floors, within which you'll decorate the floor with windows, doors, etc. Add this code right after `building.makeFoundation().join()` and before the line `building`:

```
(1..floors).forEach {
    // A floor should be raised before we can decorate it
    building.buildFloor(it).join()

    // These decorations could be made at the same time
    building.placeWindows(it)
    building.installDoors(it)
    building.provideElectricity(it)
    building.fitOut(it)
}

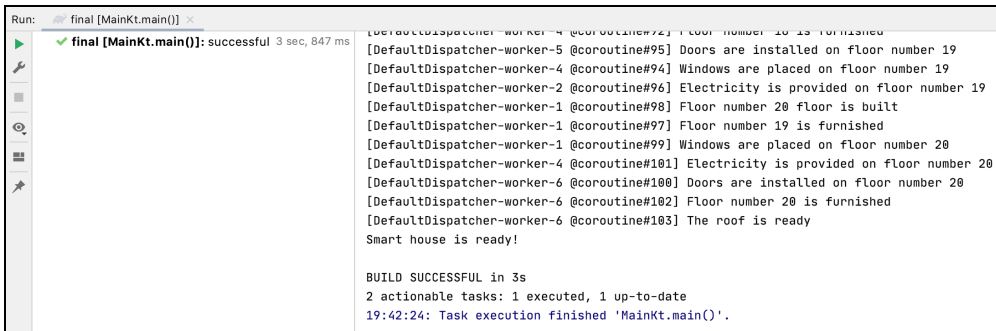
building.buildRoof().join()
```

Inside the loop over the floors, before decorating a floor, it's vital to build it, so you use the `join()` function on `building.buildFloor(it)` to wait. After that, all decorative tasks can be performed simultaneously so there's no need to suspend the current coroutine.

When all the floors are ready, you can build the final part of your building — a roof.

Build and run the latest version of your program.

You'll see that the construction of your building performs successfully:



```
Run: final [MainKt.main()] x
[DefaultDispatcher-worker-4 @coroutine#95] Floor number 19 is furnished
[DefaultDispatcher-worker-5 @coroutine#95] Doors are installed on floor number 19
[DefaultDispatcher-worker-4 @coroutine#94] Windows are placed on floor number 19
[DefaultDispatcher-worker-2 @coroutine#96] Electricity is provided on floor number 19
[DefaultDispatcher-worker-1 @coroutine#98] Floor number 20 floor is built
[DefaultDispatcher-worker-1 @coroutine#97] Floor number 19 is furnished
[DefaultDispatcher-worker-1 @coroutine#99] Windows are placed on floor number 20
[DefaultDispatcher-worker-4 @coroutine#101] Electricity is provided on floor number 20
[DefaultDispatcher-worker-6 @coroutine#100] Doors are installed on floor number 20
[DefaultDispatcher-worker-6 @coroutine#102] Floor number 20 is furnished
[DefaultDispatcher-worker-6 @coroutine#103] The roof is ready
Smart house is ready!

BUILD SUCCESSFUL in 3s
2 actionable tasks: 1 executed, 1 up-to-date
19:42:24: Task execution finished 'MainKt.main()'.
```

If you look in detail at the output, you see that your program executed the coroutines on different threads within the `CommonPool`. Also, the construction process of different floors overlaps, just like in real life (e.g., when the building of the 20th floor is started, the decorating of the 19th wasn't finished yet).

Error handling

The common approach to handle exceptions while using coroutines is a well-known try-catch block. The way you catch exceptions in synchronous code is still applicable here:

```
try {
    val userProfile = scope.withContext(Dispatchers.IO) {
        getProfile()
    }
} catch (e: NoSuchUserException) {
    // handle exception
}
```

Using CoroutineExceptionHandler

There could be a case when you need to have a global exception handler for all your coroutines, and `CoroutineExceptionHandler` is designed for this purpose:

```
val scope = CoroutineScope(Dispatchers.Default)
val handler = CoroutineExceptionHandler { context, exception ->
    println(exception.message)
}
scope.launch(handler) {
    uploadData()
}
```


Note: `CoroutineExceptionHandler` won't be triggered if it's not set to the scope of the parent coroutine, as it's supposed to be used for global handling of unexpected exceptions.

Understanding coroutines

Coroutines aren't a new concept in software development; several programming languages — such as C#, Ruby and Python — have supported them for a long time. In many languages, coroutines are based on *state machines*, and Kotlin isn't an exception.

The Kotlin compiler generates a class that represents a state machine for each of your coroutines. When your coroutine execution reaches the suspension point (i.e., invocation of a suspending function), its state machine stores the current state of the coroutine in order to easily resume the execution later. In this way, coroutines are extremely efficient, since they don't block threads and they require only one class for the execution of each of them, which is cheap and lightweight at the same time.

Challenges

Challenge 1

Modify the `BuildingYard` class in such way that you could build several buildings simultaneously, not one by one. (Hint: Consider using `Collection<Deferred<T>>.awaitAll()`)

Challenge 2

Modify the `Building` class in such way so the `buildFloor()` function could fail randomly (i.e., throw an exception). In the `BuildingYard` class, after this function execution completes, check whether it executed successfully. If it is unsuccessful, start the execution of the task again.

Key points

- The **asynchronous** approach to programming focuses on allowing you to execute several operations at the same time.
- **Threads** are used when you don't need a lot of them to perform the necessary tasks.
- **Coroutines** are like “lightweight threads”, since they don't require as much memory resources and they're not based on OS level threads like Java threads.
- A large number of coroutines could be executed on a single thread without blocking it.
- Each coroutine is bound to some `CoroutineContext`.
- `CoroutineContext` is responsible for many important parts of a coroutine such as its `Job`, `Dispatcher` and `CoroutineExceptionHandler`.
- Use coroutines builders (`runBlocking()`, `withContext()`, `launch()`, `async()`) to create and launch coroutines.

- You can decide when to launch your coroutine using `CoroutineStart`.
- Use **dispatchers** to define the threads for your coroutine execution.
- Coroutines are based on the concept of a state machine, with each state referring to a suspension point. It doesn't require extra time or resources to switch between coroutines and to restore their state.

Where to go from here?

To continue your coroutines learning journey you may find it exciting to get to know some more advanced topics, like **Channels** and **Flows**. The official documentation would be a great departing point! Those will help you to make use of *streams* of data, not just a single value. You can learn about Channels at <https://kotlinlang.org/docs/reference/coroutines/channels.html> and Flows at <https://kotlinlang.org/docs/reference/coroutines/flow.html>.

Moreover, there are several ways you could parallelize your code execution in Kotlin. One example is the reactive approach, which is becoming quite popular. **ReactiveX** or **Rx** is an API for asynchronous programming implemented by a wide variety of platforms and programming languages (e.g., Kotlin, Java, Swift, Python, etc.).

You can find out more about by reading the official RxKotlin documentation at <https://github.com/ReactiveX/RxKotlin>.

But the existence of different solutions doesn't mean that you need to choose only one. Kotlin coroutines and RxKotlin can successfully coexist in your project, as the APIs are designed to solve somewhat different programming problems. In different parts of your application, you can select the most appropriate one.

Also, you may want to get acquainted with other coroutine APIs available, such as:

- **kotlinx-coroutines-rx**: <https://github.com/Kotlin/kotlinx.coroutines/tree/master/reactive/kotlinx-coroutines-rx2>
- **kotlinx-coroutines-jdk8**: <https://github.com/Kotlin/kotlinx.coroutines/tree/master/integration/kotlinx-coroutines-jdk8>

In the next chapter, you'll have a chance to investigate the use of Kotlin away from the JVM, as a scripting tool.